statistics

R^2 – coefficient of multiple determination (förklaringsgrad)

α – confidence level

SSR – sum of squares regression

χ^2 – chi-squared – is Y normal around regression line?

numerical

SSE – sum of squared errors

MSE – mean squared errors

$$MSE = Bias^2 + var + noise$$

overfit if too high!

$$C(\vec{\theta}) = \underbrace{\frac{1}{n}\sum_{i=1}^{n}(y-\hat{y})^2}_{MSE} + \frac{\alpha}{m}\sum_{j=1}^{m}\theta_j^2$$

<u>Ridge regression</u> aka Thikhonov regularization

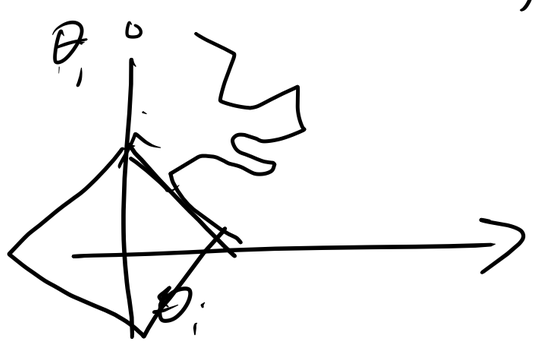
Bias increases while variance decreases

We expect this regression to generalize better, since we "overfit less" to the training data.

α - hyperparameter that indicates "strength" of regularization.

$$C(\vec{\theta}) = MSE + 2\alpha \sum_{i=1}^{m} |\theta_i|$$

LASSO-regression



Tends to set some parameters to zero.

Parameters with low significance tend towards zero.

$$C(\vec{\theta}) = MSE + r\left(2\alpha \sum_{i=1}^{m} |\theta_i| + (1-r)\left(\frac{\alpha}{m}\sum_{i=1}^{m}\theta_i^2\right)\right)$$

Elasticnet regression

r - hyperparameter that decides how much of l_1 or l_2 regularization we use.

l_1-norm = LASSO (feature selection)

l_2-norm = Ridge (generalization)

Since we don't have (good) statistics, we must use train/test splits.

If test data ends up in training, we have <u>Data Leakage</u>, which leads to strong overfit.

Other kinds of data leakage include <u>confirmation bias</u>, eg the experimenter peeks at test data and makes adjustments.

Data Leakage is particularly common / bad in fintech (ie stock-data). Eg. S&P is delayed, so including everything actually is a data leakage.

$$\hat{\theta} = (X^T X + \alpha X)^{-1} X^T y \qquad \text{(Ridge regression)}$$

Even though we <u>could</u> continue with statistics, we typically solve the minimization problem (ie $\min(C(\vec{\theta}))$), in general we use Stochastic Gradient Descent and MLE.

C.f <u>logistic loss function</u>

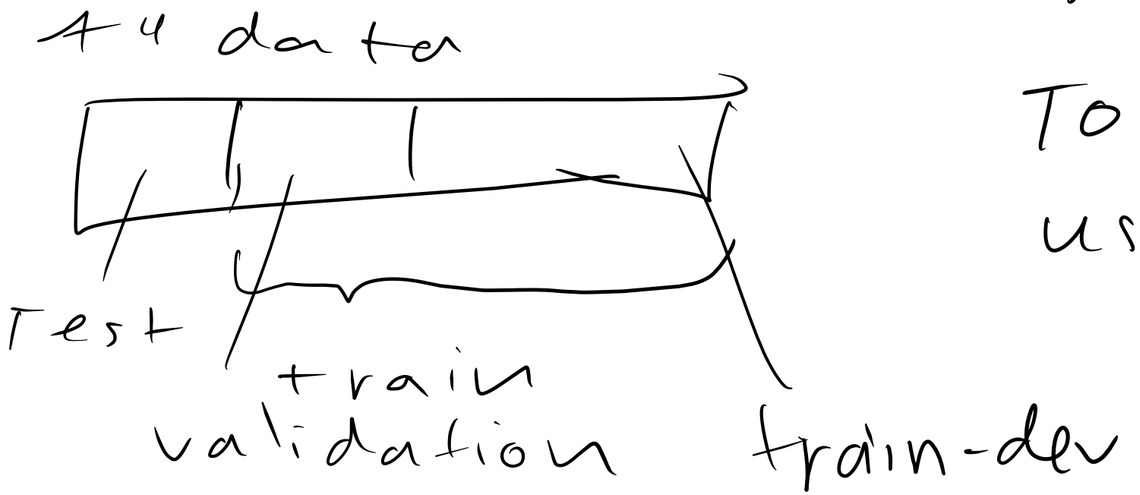
$$C(\theta) = -\frac{1}{m} \sum_{i=1}^{m} \left[y_i \ln(\hat{p}_i) + (1 - y_i) \ln(1 - \hat{p}_i) \right] + \frac{\alpha}{m} \sum \theta^2$$

$$\hat{p}(x) = \frac{1}{1 - e^{(\theta^T x)}}$$

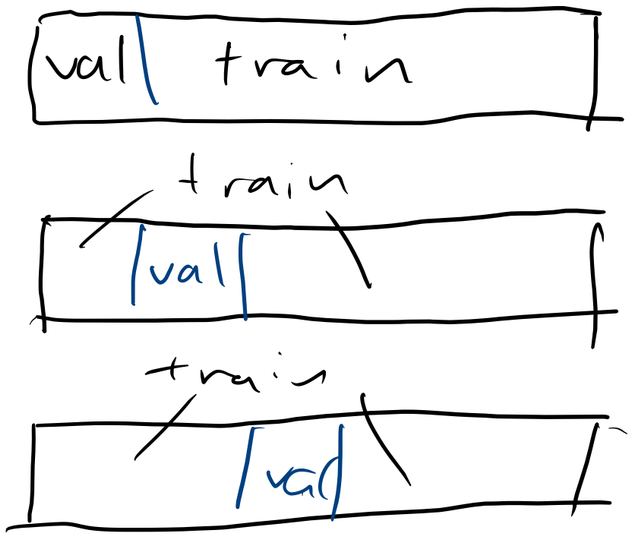
Hyper parameter optimization

Thus far, we have optimized 'k' in kNN.

To avoid data leakage we need a validation set.



To avoid random skews, use k-fold cross-validation



Train-set

iteration 1 → score

iteration 2 → score

iter. 3

iter. k+1

k-fold CV splits training data in k non-overlapping subsets, called <u>folds</u>, and we report the <u>average score</u>